Navigating the LLM Serving Trilemma: A Critical Review of Memory Bandwidth, Latency, and Co-Design Strategies

---

The deployment of Large Language Models (LLMs) is currently constrained by a critical trilemma involving memory bandwidth saturation, sequential latency bottlenecks, and escalating computational costs. This review synthesizes contemporary research to analyze the fundamental "memory wall" created by linearly growing Key-Value (KV) caches during autoregressive decoding [Keep the Cost Down: A Review on Methods to Optimize LLM's KV Cache Consumption]. We examine the dichotomy between lossy compression strategies, which risk accuracy degradation in complex reasoning [122], and hierarchical memory management, which often shifts bottlenecks to interconnect bandwidth [216]. By contrasting approaches such as coupled quantization [288] with GPU-centric storage stacks [51], we highlight inherent trade-offs between throughput and fidelity. Furthermore, we evaluate the efficacy of speculative decoding and disaggregated architectures, noting contradictions where phase separation fails under heavy offloading loads. Ultimately, this document argues that overcoming current serving limitations requires holistic algorithm-hardware co-design that dynamically balances sparsity, adaptive scheduling, and heterogeneous memory hierarchies to navigate the competing demands of modern inference workloads.

Introduction to Efficient LLM Serving Challenges and Paradigms

The deployment of Large Language Models (LLMs) in production environments is currently constrained by a critical trilemma involving memory bandwidth saturation, sequential latency bottlenecks, and escalating computational costs. The fundamental architecture of Transformers relies on Key-Value (KV) caches to store intermediate states during autoregressive decoding, converting the time complexity of token generation from quadratic to linear [Keep the Cost Down: A Review on Methods to Optimize LLM's KV Cache Consumption]. However, this optimization introduces a severe "memory wall," as the KV cache size grows linearly with sequence length, often exceeding the model's parameter count and saturating GPU High Bandwidth Memory (HBM) [225]. For instance, processing long sequences with large batch sizes can result in KV cache footprints that dwarf the model weights, creating a primary bottleneck for serving concurrency and context length [137]. This memory pressure is exacerbated by the distinct characteristics of the inference phases: while the prefill phase is typically compute-bound, the decode phase is inherently memory-bound, requiring frequent access to the growing KV cache [86].

To mitigate these constraints, the literature presents a dichotomy between compression strategies and hierarchical memory management, each introducing unique trade-offs between accuracy, latency, and hardware utilization. Compression techniques, such as quantization and token eviction, aim to reduce the physical footprint of the KV cache within GPU memory. Methods like coupled quantization have demonstrated the ability to compress KV caches down to extreme bit-widths, such as 1-bit per channel, by exploiting inter-channel dependencies [288]. Similarly, hybrid approaches combine eviction with low-bit representations to maintain accuracy while reducing size [225]. However, these lossy methods risk irreversible information forgetting, which can degrade performance in complex reasoning tasks or long-context retrieval [122]. Furthermore, the overhead of online outlier detection and dynamic compression can negate latency gains, necessitating co-designed hardware accelerators or offline calibration strategies [2].

In contrast to compression, hierarchical memory management and offloading strategies seek to expand effective capacity by utilizing CPU DRAM, SSDs, or distributed nodes. While offloading enables the serving of contexts that far exceed GPU limits, it shifts the bottleneck from memory capacity to interconnect bandwidth. Analytical frameworks reveal that for workloads where cached tokens significantly outnumber new tokens, PCIe bandwidth limitations can cause execution to become entirely memory-bound, leaving GPU compute resources underutilized by as much as 72% [216]. The fragmentation inherent in paged memory layouts, a standard feature of modern serving engines like vLLM, further degrades I/O efficiency by generating massive numbers of tiny random reads when restoring caches from slower storage tiers [51]. To address this, recent systems propose GPU-centric storage stacks that eliminate CPU intervention from the critical I/O path, enabling bulk transfers that saturate NVMe bandwidth [51]. Additionally, emerging tightly coupled architectures, such as Superchips with NVLink-C2C interconnects, offer an order of magnitude higher bandwidth than PCIe, yet require specialized SLO-aware scheduling to fully exploit their potential for responsive serving [75].

The tension between theoretical compression ratios and practical serving constraints is further complicated by the heterogeneity of modern model architectures and workloads. Non-uniform attention patterns across different heads suggest that monolithic cache management is inefficient, prompting systems that allocate memory budgets at the head level or employ hierarchical caching based on temporal stability [96] [269]. Moreover, the rise of multi-turn conversations and Retrieval-Augmented Generation (RAG) introduces prefix reuse opportunities that are often squandered by rigid scheduling policies or insufficient cache retention strategies [24]. While some approaches advocate for disaggregating prefill and decode phases to optimize hardware specialization, this strategy falters when offloading renders both phases memory-bound, undermining the benefits of hardware heterogeneity [216]. Consequently, the field is moving toward holistic co-designs that integrate algorithmic sparsity, adaptive scheduling, and hardware-aware memory hierarchies to navigate the competing demands of throughput, latency, and cost [169].

The Memory Bandwidth Bottleneck and Decode Latency

The fundamental constraint governing modern Large Language Model (LLM) inference has undergone a paradigmatic shift from compute-bound operations to memory-bound data movement, a transition that is most acute during the autoregressive decoding phase. In traditional transformer architectures, the decoding process requires attending to the entire history of generated tokens, necessitating the storage and retrieval of Key-Value (KV) caches that grow linearly with sequence length. As context windows expand to accommodate hundreds of thousands or even millions of tokens, the memory footprint of these caches frequently exceeds the capacity of on-chip High Bandwidth Memory (HBM), forcing systems to rely on slower off-chip DRAM or host memory [216]. This reliance creates a severe bandwidth mismatch; while GPUs offer terabytes per second of HBM bandwidth, the interconnects linking GPU to CPU, such as PCIe, provide only tens of gigabytes per second. Consequently, empirical characterizations reveal that in offloading scenarios, up to 99% of latency can be attributed to data transfers, leaving GPU compute resources underutilized and consuming a fraction of their rated thermal design power [216].

The severity of this bottleneck is quantified by the ratio of cached tokens to newly prefilled tokens. Analytical frameworks define a critical threshold, κ_{crit}, beyond which execution transitions from compute-bound to memory-bound. Typical workloads, particularly those involving long-document queries or multi-turn conversations, exceed this threshold by orders of magnitude, with median ratios reaching 5000 for document analysis tasks [216]. This phenomenon renders the prefill phase, traditionally compute-intensive, effectively memory-bound when large KV caches must be retrieved from slower storage tiers. The issue is exacerbated by the fragmented nature of paged memory management used in modern serving engines like vLLM and SGLang, which breaks logically contiguous KV caches into small, scattered blocks. When restoring these caches from SSDs or DRAM, this fragmentation generates massive numbers of tiny random I/O operations, creating a CPU-centric bottleneck that induces significant GPU stalls even when using technologies like GPU Direct Storage [51].

To mitigate these bandwidth and capacity constraints, the literature presents a diverse array of strategies ranging from hardware co-design to algorithmic compression. On the hardware front, emerging architectures such as NVIDIA's Grace Hopper Superchips utilize NVLink-C2C interconnects to provide an order of magnitude higher bandwidth than PCIe, enabling more efficient CPU-GPU offloading. However, realizing these gains requires software stacks specifically designed to exploit full-duplex transfers and proactive scheduling, as naive porting of PCIe-based mechanisms fails to utilize the available bandwidth [75]. Similarly, rack-scale solutions leveraging Compute Express Link (CXL) shared memory aim to eliminate network hops in disaggregated serving, allowing direct load/store access to remote KV caches [68]. Despite these advancements, the cost and scaling limitations of HBM continue to drive research into alternative memory hierarchies, including the use of eDRAM for edge devices and flash memory for capacity expansion, though these introduce their own latency and refresh overheads [121; 200].

Algorithmic approaches focus on reducing the volume of data movement through compression and selective attention. Quantization remains a primary technique, with methods evolving from uniform low-bit representations to sophisticated mixed-precision and outlier-aware schemes that preserve accuracy while drastically reducing cache size [2; 277]. More aggressive compression techniques, such as transform coding and joint encoding, exploit redundancies across attention heads and tokens to achieve compression ratios exceeding 20x with negligible accuracy loss [107; 23]. Complementing compression, sparse attention mechanisms and token eviction policies aim to reduce the working set size by retaining only the most critical tokens or attending to a subset of the context [242; 269]. However, these methods often face a trade-off between memory savings and the complexity of irregular memory access patterns, which can undermine hardware efficiency if not carefully managed [55].

A critical contradiction exists between the goal of maximizing cache reuse through prefix caching and the physical limitations of memory capacity. While prefix caching eliminates redundant computation for shared prompts, the accumulation of unique suffixes in concurrent sessions rapidly exhausts GPU memory, leading to head-of-line blocking and Service Level Objective (SLO) violations [43]. Systems attempting to resolve this via hierarchical storage often struggle with the latency of retrieving fine-grained data from lower tiers, prompting innovations in GPU-centric I/O stacks that bypass CPU intervention entirely [51]. Furthermore, the assumption that prefill and decode phases have distinct resource profiles is challenged in long-context regimes where both become bandwidth-limited, undermining the efficacy of disaggregated architectures that assign these phases to different hardware specialized for compute or

memory [216]. Ultimately, addressing the memory bandwidth bottleneck requires a holistic co-design of memory controllers, interconnects, scheduling algorithms, and compression techniques to balance the competing demands of capacity, bandwidth, and latency in an era of exponentially growing context lengths [167; 190].

Taxonomy of Optimization Strategies

To address the critical bottlenecks imposed by the linearly growing memory footprint of Key-Value (KV) caches in Large Language Model (LLM) serving, contemporary research has diverged into three primary categories: KV cache compression and management, speculative decoding to mitigate sequential dependencies, and quantization to reduce the memory footprint. This taxonomy distinguishes between lossless approaches, which preserve exact model outputs through mechanisms like speculative verification and exact prefix caching, and lossy methods, such as aggressive quantization and token pruning, which trade marginal accuracy for significant gains in latency and throughput. The interplay between these strategies defines the current landscape of efficient LLM serving, where system designers must navigate complex trade-offs between memory capacity, bandwidth utilization, and generation fidelity.

The first pillar, KV cache compression and management, focuses on reducing storage requirements through eviction, offloading, or structural reorganization. A fundamental distinction exists between lossy compression, which permanently discards information, and lossless management, which offloads data to slower storage tiers without information loss. Lossy methods often employ token eviction or pruning based on attention scores. For instance, Lethe: Layer- and Time-Adaptive KV Cache Pruning for Reasoning-Intensive LLM Serving introduces a dynamic framework that performs layer-wise sparsity-aware allocation and recency-aware selective retention to prune tokens during generation. Similarly, Keyformer: KV Cache Reduction Through Key Tokens Selectionfor Efficient Generative Inference retains only "key" tokens that account for the majority of attention weight, significantly reducing memory bandwidth usage. However, these approaches risk catastrophic failures in tasks requiring long-range dependency, as noted in VeriCache: Turning Lossy KV Cache into Lossless LLM Inference, which highlights that lossy outputs increasingly diverge from full-cache baselines as sequence length grows. To mitigate this, EVICPRESS: Joint KV-Cache Compression and Eviction for Efficient LLM Serving proposes a unified utility function to jointly optimize compression and eviction decisions across storage tiers, balancing quality and delay. Conversely, lossless management strategies leverage hierarchical memory systems. SpeCache: Speculative Key-Value Caching for Efficient Generation of LLMs utilizes CPU memory to store the complete KV cache, employing speculative prediction to prefetch relevant KV pairs into GPU VRAM before they are needed, thereby hiding PCIe latency. Extending this to deeper storage hierarchies, Tutti: Making SSD-Backed KV Cache Practical for Long-Context LLM Serving eliminates CPU intervention in the I/O path by introducing a GPU-centric object store and asynchronous GPU io_uring, saturating NVMe bandwidth and reducing GPU stalls. Furthermore, CacheFlow: Efficient LLM Serving with 3D-Parallel KV Cache Restoration rethinks restoration as a multi-dimensional parallel execution problem, overlapping recomputation and I/O across tokens, layers, and GPUs. These systems often rely on sophisticated reuse mechanisms; MEPIC: Memory Efficient Position Independent Caching for LLM Serving enables chunk-level KV reuse across arbitrary positions and requests by aligning chunks to paged storage and fusing Rotary Position Embeddings, while SparseX: Efficient Segment-Level KV Cache Sharing for Interleaved LLM Serving exploits segment-level sharing for non-prefix content in interleaved workloads.

The second category, speculative decoding, addresses the sequential bottleneck of autoregressive generation by decoupling token drafting from verification. Traditional speculative decoding employs a small draft model to generate candidate tokens, which are then verified in parallel by the target model, as foundational work in Fast Inference from Transformers via Speculative Decoding and Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation demonstrates. Recent advancements have evolved this paradigm to enhance parallelism and adaptability. CARD: Cache-Assisted Parallel Speculative Decoding for Efficient Large Language Model Inference decouples drafting and verification into a "query-and-correct" paradigm, allowing the target model to rectify draft directions concurrently rather than sequentially. To address the variability of workloads, Nightjar: Dynamic Adaptive Speculative Decoding for Large Language Models Serving introduces a resource-aware framework that dynamically adjusts speculation length and disables speculation when it becomes detrimental to throughput in compute-bound environments. Moreover, MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding challenges the convention that speculative decoding is ineffective at high batch sizes, showing that intelligent drafting with sparse KV caches can yield speedups even in high-throughput regimes. Self-speculative approaches, such as Cassandra: Enabling Reasoning LLMs at Edge via Self-Speculative Decoding and Quasar: Quantized Self-Speculative Acceleration for Rapid Inference via Memory-Efficient Verification, further optimize this

by using the target model itself or quantized versions thereof for drafting, reducing the need for separate draft models and minimizing memory traffic during verification.

The third strategy, quantization, aims to reduce the memory footprint of the KV cache by lowering the precision of stored keys and values. While naive low-bit quantization often leads to significant accuracy degradation due to outliers, recent co-design approaches have achieved near-lossless performance. SAW-INT4: System-AWare 4-Bit KV-Cache Quantization for Real-World LLM Serving identifies that token-wise INT4 quantization with block-diagonal Hadamard rotation offers the optimal trade-off under real-world serving constraints, recovering accuracy lost by naive methods without sacrificing throughput. More aggressive compression is explored in H1B-KV: Hybrid One-Bit Caches for Memory-Efficient Large Language Model Inference, which combines 1-bit binary sketches for keys with 4-bit quantization for values, achieving a 70x memory reduction. Advanced error correction mechanisms are also employed; GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM augments quantization with low-rank and sparse matrices to remedy outlier errors, while VecInfer: Efficient LLM Inference with Low-Bit KV Cache via Outlier-Suppressed Vector Quantization uses smooth and Hadamard transformations to suppress outliers before vector quantization. Mixed-precision strategies, such as those in Cocktail: Chunk-Adaptive Mixed-Precision Quantization for Long-Context LLM Inference and MoQAE: Mixed-Precision Quantization for Long-Context LLM Inference via Mixture of Quantization-Aware Experts, dynamically assign bit-widths based on chunk similarity or expert routing to maintain accuracy while minimizing memory usage. Finally, XQUANT: Breaking the Memory Wall for LLM Inference with KV Cache Rematerialization takes a distinct approach by quantizing layer input activations and rematerializing keys and values on-the-fly, trading increased computation for drastic memory savings.

These three categories are not mutually exclusive but are increasingly integrated to maximize efficiency. For example, VeriCache: Turning Lossy KV Cache into Lossless LLM Inference combines lossy compression with speculative verification to ensure exact outputs, while Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization co-designs algorithm and hardware to optimize quantization thresholds. The convergence of these strategies suggests that future LLM serving systems will rely on hybrid architectures that dynamically switch between compression, speculation, and quantization based on workload characteristics and hardware constraints.

Evaluation Metrics and Trade-offs

Effective optimization of Large Language Model (LLM) serving necessitates a rigorous balancing act between latency metrics, specifically Time-to-First-Token (TTFT) and Time-Per-Output-Token (TPOT), system throughput, and model accuracy. The literature reveals that these objectives are often mutually exclusive, requiring sophisticated trade-off management strategies tailored to specific hardware constraints and workload characteristics. A primary metric for evaluating quantization methods is perplexity degradation, which serves as a proxy for accuracy loss when compressing the Key-Value (KV) cache. While aggressive quantization reduces memory footprint, it frequently incurs accuracy penalties that vary significantly based on the compression strategy employed. For instance, [2] demonstrates that hybrid online-offline approaches can achieve substantial throughput improvements with minimal accuracy loss, whereas naive low-bit quantization often fails to preserve model fidelity. Similarly, [226] argues that under real-world serving constraints, simple token-wise INT4 quantization with block-diagonal Hadamard rotation offers the optimal accuracy-efficiency trade-off, outperforming more complex vector quantization methods that provide marginal gains at the cost of system compatibility.

The tension between memory capacity and computational latency is further highlighted by approaches that prioritize different aspects of the serving pipeline. [277] and [215] both advocate for mixed-precision strategies to handle long contexts, yet they differ in their granularity; Cocktail operates at the chunk level to mitigate hardware inefficiency, while MoQAE utilizes a mixture-of-experts router to select bit-widths dynamically. In contrast, [201] proposes rematerializing keys and values from quantized activations, trading increased computation for an order-of-magnitude reduction in memory consumption, thereby challenging the conventional reliance on storing full KV caches. This compute-memory trade-off is also central to [122], which ensures lossless output by verifying compressed cache drafts against a full cache stored off-GPU, effectively swapping memory bandwidth for verification overhead.

Speculative decoding introduces a distinct set of metrics, primarily the acceptance rate of draft tokens, which directly dictates the speedup potential. However, the efficacy of speculation is highly dependent on the system's ability to manage the verification bottleneck. [111] addresses the memory-bound nature of the verification phase by applying low-bit quantization specifically to this stage, preserving acceptance lengths while halving memory traffic. Conversely, [11] highlights a critical contradiction in speculative decoding: while it boosts throughput in low-load, memory-bound scenarios, it can degrade performance in high-load, compute-bound environments due to verification overhead. Nightjar proposes dynamically disabling speculation when it ceases to be beneficial, illustrating that static speculative strategies often fail to adapt to fluctuating workload demands. Furthermore, [87] decouples drafting and verification to allow parallel execution, challenging the traditional sequential "draft-then-verify" paradigm that limits draft model size and efficiency.

Scheduling policies and cache management systems introduce additional trade-offs between fairness, tail latency, and overall throughput. [43] identifies that rigid First-Come-First-Serve (FCFS) scheduling leads to suboptimal batch composition and TTFT SLO violations. By employing a hybrid cache scheme and adaptive scheduling, Apt-Serve significantly improves effective throughput, suggesting that dynamic batch composition is superior to static policies. This finding is supported by [213], which theoretically proves that standard heuristics like FCFS and Shortest-First have unbounded competitive ratios in the presence of variable input sizes, necessitating algorithms with provable guarantees like Sorted-F. Meanwhile, [168] focuses on tail latency, proposing proactive cache allocation and preemption strategies to reduce tail TTFT and TBT, arguing that average throughput metrics often mask severe user experience degradation in time-sensitive applications.

Energy efficiency emerges as another critical dimension, often overlooked in favor of pure speed metrics. [79] reveals that lower-precision formats only yield energy gains in compute-bound regimes, while batching is more effective in memory-bound phases. This suggests that optimization strategies must be phase-aware, as an approach that maximizes throughput may not minimize energy consumption. Similarly, [170] emphasizes that naive estimates based on FLOPs underestimate real-world energy use, calling for workload-aware optimization. Finally, the emergence of specialized hardware and data formats, such as the Anda data type in [257] and the ReRAM-based accelerator in [156], indicates that future trade-offs will increasingly depend on algorithm-hardware co-design, where metrics like area efficiency and energy per token become as vital as traditional latency measures.

Advanced KV Cache Management and Compression Techniques

Efficient management of the Key-Value (KV) cache has emerged as the primary bottleneck in scaling Large Language Model (LLM) serving, particularly as context windows expand to hundreds of thousands of tokens. The linear growth of KV cache memory with sequence length necessitates a divergence in research strategies, broadly categorized into quantization, sparsity, intelligent eviction, and hierarchical storage management. A central consensus across recent literature is that aggressive compression algorithms must be co-designed with existing serving infrastructure, such as paged attention mechanisms, to yield tangible performance benefits without compromising model fidelity [137].

Quantization remains a dominant strategy for reducing the memory footprint of KV caches, yet the trade-off between bit-width reduction and accuracy preservation requires sophisticated handling of outlier values. Traditional approaches often employ mixed-precision schemes, retaining higher bit-widths for outliers while quantizing the majority of values to low bit-widths like INT4. However, Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization argues that the computational cost of online outlier detection can negate the latency benefits of quantization. To resolve this, Oaken proposes a hybrid approach where outlier thresholds are determined offline, allowing for efficient online scaling with minimal accuracy loss. Similarly, SAW-INT4: System-AWare 4-Bit KV-Cache Quantization for Real-World LLM Serving emphasizes that system compatibility is paramount; it demonstrates that a simple token-wise INT4 quantization with block-diagonal Hadamard rotation outperforms more complex vector quantization methods when integrated into paged memory layouts, as the latter often disrupt regular memory access patterns required by fused attention kernels. Pushing the boundaries of compression further, H1B-KV: Hybrid One-Bit Caches for Memory-Efficient Large Language Model Inference introduces a hybrid scheme utilizing 1-bit binary sketches for keys and 4-bit quantization for values, achieving drastic memory reductions while maintaining performance on complex reasoning tasks through lightweight fine-tuning. Complementing these efforts, GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM and KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference explore error correction via low-rank approximation and layer-wise sensitivity analysis, respectively, to achieve near-lossless inference at high compression ratios.

Beyond quantization, research has increasingly focused on sparsity and intelligent eviction policies to retain only the most "informative" tokens. KeyFormer: KV Cache Reduction Through Key Tokens Selection for Efficient Generative Inference operates on the observation that attention weights are heavily concentrated on a small subset of key tokens, allowing for the eviction of less critical entries without significant accuracy degradation. This concept is extended by FlexiCache: Leveraging Temporal Stability of Attention Heads for Efficient KV Cache Management, which classifies attention heads based on the temporal stability of their focus, offloading stable heads to host memory while keeping unstable heads on the GPU. However, lossy compression methods inherently risk output divergence over long generation horizons. VeriCache: Turning Lossy KV Cache into Lossless LLM Inference addresses this by introducing a verification framework where compressed caches draft tokens that are subsequently verified against a full KV cache stored in lower-tier memory, ensuring identical outputs to full-precision decoding while preserving high throughput.

The management of KV caches across heterogeneous memory hierarchies represents another critical frontier. As GPU High-Bandwidth Memory (HBM) becomes exhausted, systems must efficiently offload data to CPU DRAM or SSDs. EVICPRESS: Joint KV-Cache Compression and Eviction for Efficient LLM Serving proposes a unified utility function to jointly optimize compression and eviction decisions across storage tiers, maximizing hit rates on fast devices. Meanwhile, Tutti: Making SSD-Backed KV Cache Practical for Long-Context LLM Serving identifies CPU intervention as a major bottleneck in SSD-backed caching and presents a GPU-centric object store that eliminates CPU involvement in the I/O control path, thereby saturating NVMe bandwidth. In distributed settings, ShadowServe: Interference-Free KV Cache Fetching for Distributed Prefix Caching leverages SmartNICs to offload data plane operations, preventing interference with host computation during prefix cache fetches. Furthermore, SpeCache: Speculative Key-Value Caching for Efficient Generation of LLMs utilizes speculative prediction to prefetch relevant KV pairs from CPU memory to GPU VRAM, effectively hiding the latency of PCIe transfers.

System-level innovations also address the structural inefficiencies of current caching abstractions. RedKnot: Efficient Long-Context LLM Serving with Head-Aware KV Reuse and SegPagedAttention challenges the monolithic view of KV caches, proposing a head-aware decomposition that allows for differentiated management policies based on the

functional roles of specific attention heads. Similarly, Tangram: Unlocking Non-Uniform KV Cache for Efficient Multi-turn LLM Serving tackles the memory fragmentation caused by non-uniform compression through deterministic budget allocation and head grouping. For multi-tenant and multi-agent scenarios, KVShare: An LLM Service System with Efficient and Effective Multi-Tenant KV Cache Reuse and TokenDance: Scaling Multi-Agent LLM Serving via Collective KV Cache Sharing introduce mechanisms to share KV caches across requests and agents, significantly reducing redundancy in collaborative workflows. Finally, MEPIC: Memory Efficient Position Independent Caching for LLM Serving enhances prefix caching by enabling chunk-level reuse at arbitrary positions through page-aligned storage and fused Rotary Position Embedding (RoPE) attention, overcoming the strict prefix-matching limitations of earlier systems. These diverse approaches collectively underscore that future advancements in LLM serving depend on a holistic co-design of algorithms, memory hierarchies, and hardware accelerators.

Bit-Level Quantization and Hybrid Caching

Quantizing the Key-Value (KV) cache to low bit-widths stands as a primary defense against the severe memory pressure inherent in large language model serving, particularly as context lengths scale to hundreds of thousands of tokens. However, the efficacy of aggressive quantization is frequently compromised by activation outliers, which induce significant accuracy degradation when naive low-bit schemes are applied. Recent research has converged on system-aware co-design approaches that integrate algorithmic robustness with hardware constraints to mitigate these issues. A pivotal finding in this domain is that token-wise INT4 quantization, when combined with block-diagonal Hadamard rotation, offers an optimal trade-off between accuracy and efficiency for real-world deployment. [226] demonstrates that this specific configuration recovers nearly all accuracy lost by naive INT4 quantization while introducing zero measurable overhead in paged memory layouts. The study emphasizes that more complex methods, such as vector quantization or Hessian-aware strategies, provide only marginal gains once strict serving constraints like fused attention execution and regular memory access are enforced. This suggests that for real-world deployment, lightweight rotation techniques are superior to computationally heavy alternatives when operating within standard paged attention frameworks.

Contrasting with the simplicity of rotation-based INT4, other works explore vector quantization to achieve even lower bit-widths, such as 2-bit, though this requires sophisticated outlier suppression to prevent codebook collapse. [26] addresses the failure of standard vector quantization at ultra-low bit-widths by applying smooth and Hadamard transformations to suppress key cache outliers. This allows the codebook to comprehensively cover the data distribution, enabling 2-bit quantization that matches full-precision performance while delivering substantial speedups in self-attention computation. Similarly, [36] proposes a hybrid error reduction framework that combines ultra-low precision quantization for the majority of entries with a low-rank matrix to approximate quantization errors and a sparse matrix to remedy individual outlier errors. This multi-component approach achieves near-lossless performance at high compression ratios, outperforming state-of-the-art baselines that rely solely on group-wise quantization or token dropping. These methods illustrate that while simple quantization fails at extreme compression, augmenting it with error correction mechanisms can restore fidelity.

The challenge of handling outliers has also led to the development of mixed-precision and hybrid caching strategies that dynamically allocate bit-widths based on token or chunk importance. [2] introduces an online-offline hybrid approach where outlier thresholds are determined offline to guide online quantization scales, avoiding the high cost of real-time outlier detection. This co-design of algorithm and hardware allows Oaken to achieve high throughput with minimal accuracy loss. In a similar vein, [277] optimizes KV cache management by determining optimal bit-width configurations at the chunk level rather than the token level. By reordering KV cache chunks before quantization, Cocktail avoids the hardware inefficiencies typically associated with mixed-precision computation, thereby maintaining high accuracy while reducing memory usage. Further pushing the boundaries of compression, [240] proposes a comprehensive scheme representing keys with 1-bit binary sketches and values with 4-bit quantization. This holistic approach enables a 70x memory reduction, allowing large models to handle extensive contexts within tight memory constraints without sacrificing performance on complex reasoning tasks.

Beyond static quantization schemes, recent innovations explore rematerialization and coupled quantization to break the memory wall by exploiting structural redundancies. [201] diverges from standard KV caching by quantizing and caching layer input activations instead, rematerializing Keys and Values on-the-fly. This method leverages the similarity of input values across layers to achieve up to 10x memory savings with negligible perplexity degradation, surpassing intricate outlier-aware KV quantization methods. Additionally, [288] observes that distinct channels of key/value activations are highly inter-dependent. By coupling multiple channels to exploit this joint entropy, Coupled Quantization preserves model quality even when the KV cache is quantized down to 1-bit per channel, challenging the notion that such low bit-widths are infeasible for maintaining accuracy. These approaches indicate that future compression gains may rely less on per-tensor quantization and more on exploiting cross-layer and cross-channel correlations.

System-level integration remains critical for these quantization techniques to realize their theoretical benefits, as algorithmic gains can be nullified by memory fragmentation or verification overheads. [146] highlights that quantized models often suffer from memory fragmentation due to smaller KV block sizes. FineServe addresses this with a

precision-aware adaptive memory management technique that dynamically allocates KV cache based on quantization characteristics, significantly improving throughput and SLO attainment. Furthermore, [122] offers a unique perspective by using compressed KV caches to draft tokens while verifying them against a full KV cache stored off-GPU. This approach ensures lossless output identical to full-precision decoding while preserving the high throughput of compression methods, effectively bridging the gap between efficiency and correctness. Collectively, these works illustrate that effective bit-level quantization is not merely an algorithmic problem but a systems co-design challenge requiring careful alignment of rotation techniques, mixed-precision strategies, and memory management protocols to handle activation outliers without compromising serving efficiency.

Sparsity, Token Selection, and Pruning

Within the broader domain of Advanced KV Cache Management and Compression Techniques, sparsity-based approaches represent a paradigm shift from uniform compression to selective retention. Rather than treating all tokens with equal importance, these mechanisms aim to identify and preserve only the "key" tokens that contribute most significantly to the attention output, thereby alleviating the memory bandwidth bottlenecks inherent in long-context inference. The foundational observation driving this field is that attention distributions are highly skewed; for instance, Keyformer leverages the finding that approximately 90% of attention weight focuses on a small subset of tokens, allowing the system to dynamically retain only these critical elements to reduce memory bandwidth usage by over 2x [242]. This principle of non-uniform importance suggests that aggressive pruning can be performed without substantial accuracy degradation if the selection mechanism is sufficiently precise. However, the criteria for determining token importance vary significantly across methodologies, leading to distinct algorithmic innovations and trade-offs.

A critical challenge in token selection is ensuring fairness and accuracy in scoring mechanisms, particularly within decoder structures. A2SF highlights a specific flaw in standard Accumulative Attention Scores, where masking causes uneven comparisons based on token age, leading to biased retention of newer tokens. To rectify this, A2SF introduces a "Forgetting Factor" that penalizes older tokens, ensuring a fairer comparison and improving accuracy in models like LLaMA 2 [176]. In contrast, other approaches argue that page-level retrieval is insufficient due to the sparse distribution of important tokens. FIER proposes a fine-grained retrieval method using 1-bit quantized keys to estimate individual token importance with high precision, matching full KV performance using only 11% of the cache budget [127]. Furthermore, EntmaxKV diverges from softmax-based approximations by utilizing α-entmax, which produces exact zeros, transforming sparse decoding into a support recovery problem that allows for exact sparse decoding if the entmax support is recovered [297]. While some methods accept approximate solutions to gain higher compression ratios, others like Slim Attention claim to shrink context memory by 2x through a mathematically identical implementation of standard attention, effectively making the K-cache sufficient for Multi-Head Attention without loss [174].

The implementation of sparse attention also faces significant challenges regarding the reliability of token identification and the overhead of selection processes. TidalDecode addresses the limitation where existing mechanisms fail to reliably identify relevant tokens or overlook spatial coherence across layers. By introducing position-persistent sparse attention, TidalDecode utilizes a few layers performing full attention to identify high-scoring tokens, which are then reused in subsequent layers, effectively reducing token selection overhead while maintaining generative performance [244]. Similarly, Progressive Sparse Attention (PSA) critiques the rigid trade-off between accuracy and efficiency found in fixed top-k selection methods. PSA introduces an adaptive algorithm that adjusts the KV cache budget for different tokens and layers based on real-time attention weight distributions, coupled with a pipelined iteration scheme to minimize CPU-GPU synchronization [145]. LServe further advances this by unifying static and dynamic sparsity patterns, converting half of the attention heads into streaming heads and utilizing a hierarchical KV page selection policy to accelerate both prefilling and decoding [53].

System-level integration remains a critical hurdle, as algorithmic sparsity often fails to translate into end-to-end gains due to irregular memory access patterns. SparseServe tackles the shift from bandwidth to capacity bottlenecks by implementing hierarchical HBM-DRAM management, enabling the offloading of unselected KV caches to DRAM to free up high-bandwidth memory for larger batch sizes [55]. Complementing this, Spin unifies different sparsity granularities onto a shared page-based substrate, employing a locality-aware manager to reduce PCIe round-trips and improve throughput significantly over original sparse implementations [184]. Contradictions and nuances arise when comparing layer-specific strategies. While some approaches advocate for uniform sparsity, others like Lethe and RedKnot emphasize layer-wise and head-wise heterogeneity. Lethe performs layerwise sparsity-aware allocation based on estimated attention redundancy and employs a Recency-Aware Selective Retention mechanism for temporal adaptivity [50]. RedKnot similarly decomposes the KV cache along the head dimension, observing that different heads exhibit varying functional roles and attention ranges, thus enabling head-aware management that supports position-independent reuse and distributed placement [217]. FlexiCache adds another dimension by classifying KV heads based on temporal stability, retaining all pages for unstable heads while offloading stable heads to host memory, reducing GPU footprint by up to 70% [269].

Despite the promise of these methods, tensions exist between lossless compression and approximate pruning. H1B-KV proposes a hybrid one-bit cache scheme that compresses keys to binary sketches and values to 4-bit quantization, achieving a 70x reduction in cache memory while matching full-precision performance after lightweight fine-tuning [240]. The diversity of these approaches underscores that no single sparsity strategy dominates; rather, the optimal choice depends on the specific interplay between hardware constraints, model architecture, and the required balance between latency, throughput, and accuracy.

Hierarchical Storage and Offloading Strategies

As the context windows of Large Language Models expand to encompass millions of tokens, the Key-Value (KV) cache footprint frequently exceeds the capacity of GPU High Bandwidth Memory (HBM), necessitating a shift toward hierarchical storage architectures that incorporate CPU DRAM and NVMe SSDs. While offloading to these lower tiers provides virtually infinite capacity, it introduces severe I/O latency challenges that can transform traditionally compute-bound prefill phases into memory-bound bottlenecks. The fundamental issue lies in the mismatch between the fine-grained, paged memory layouts used by modern inference engines and the coarse-grained access patterns required for efficient storage I/O. Systems like vLLM and SGLang partition KV caches into small, non-contiguous blocks to manage fragmentation, but this results in massive numbers of tiny random I/O operations when restoring contexts from SSDs, leading to significant GPU stalls and underutilization [51]. Traditional approaches relying on CPU-centric control paths, even when enhanced with GPU Direct Storage (GDS), remain limited because the CPU must initiate each individual I/O request, creating a serialization bottleneck that prevents the saturation of NVMe bandwidth [51].

To address the inefficiencies of fragmented I/O in paged layouts, recent research has focused on decoupling GPU and CPU memory layouts and employing GPU-assisted I/O mechanisms. Strata introduces a hierarchical context caching framework that specifically targets the fragmentation caused by paged layouts by decoupling the memory layouts of the GPU and CPU [152]. By employing GPU-assisted I/O, Strata balances compute operations with loading delays, effectively overlapping unavoidable stalls with complementary tasks to reduce Time-To-First-Token (TTFT) significantly compared to baseline systems [152]. Similarly, Tutti proposes a GPU-centric object store that eliminates CPU intervention from the critical data and I/O control paths [51]. By introducing a GPU-native object abstraction and a re-architected storage stack featuring GPU io_uring, Tutti enables the GPU to issue massive parallel I/O requests directly to SSDs, reducing GPU stalls to near zero and achieving inference performance comparable to DRAM-backed systems [51]. These approaches highlight a consensus that moving I/O control to the GPU and managing data in larger, bulk objects rather than small pages is essential for practical SSD-backed caching.

Beyond SSD offloading, the hierarchy often includes CPU DRAM, where the PCIe interconnect becomes the primary bottleneck. Analytical frameworks reveal that for workloads where cached tokens vastly outnumber new tokens, the ratio of offloaded to prefilled tokens exceeds the critical threshold where execution becomes memory-bound, causing GPUs to consume only a fraction of their rated power while waiting for data transfers [216]. To mitigate this, systems like SuperInfer leverage emerging Superchip architectures with tightly coupled GPU-CPU interconnects (e.g., NVLink-C2C) to achieve full-duplex data transfers, utilizing an SLO-aware rotary scheduler to proactively rotate requests between HBM and DRAM based on latency urgency [75]. In contrast, systems operating on standard PCIe architectures must rely on sophisticated scheduling and compression. AdaptCache employs lossy KV cache compression to increase the hit rate in DRAM, thereby minimizing the frequency of slow SSD loads without significantly degrading generation quality [141]. Furthermore, MultiPath Transfer Engine (MMA) addresses the bandwidth limitations of single PCIe links by enabling efficient multipath data transfer between GPU and host memory, significantly improving transfer bandwidth and reducing TTFT [210].

The challenge of hierarchical storage is further complicated by the need to maintain Service Level Objectives (SLOs) under dynamic workloads. Static offloading strategies often fail to adapt to rapidly shifting memory demands, leading to latency spikes. OrbitFlow addresses this by employing a fine-grained, adaptive KV cache management system that uses an ILP solver to decide which layers' KV caches to retain on the GPU for each request, continuously refining placements based on runtime feedback to meet latency SLOs [4]. Additionally, the integration of compression with eviction decisions offers another layer of optimization. EVICPRESS jointly optimizes eviction and compression across multiple storage tiers using a unified utility function, ensuring that KV caches are placed on the most appropriate tier (HBM, DRAM, or SSD) based on their sensitivity to compression errors and access frequency [291]. Meanwhile, ObjectCache explores storing KV caches in remote S3-compatible object storage, co-designing the storage protocol to deliver data in the order the GPU consumes it, thus overlapping transfer with compute even across network boundaries [245]. Collectively, these works demonstrate that efficient hierarchical storage requires a co-design of memory layout, I/O control paths, scheduling algorithms, and compression techniques to balance the trade-offs between capacity, latency, and throughput.

Prefix Reuse and Cross-Request Sharing

In multi-tenant large language model serving environments, the redundancy of computational work across different requests presents a significant opportunity for optimization, particularly when requests share common prefixes, templates, or context segments. The fundamental challenge lies in the fact that while prefix caching effectively handles identical leading tokens, practical applications often exhibit repeated content that appears as non-prefix, cross-request, or interleaved segments, rendering conventional cache mechanisms insufficient. To address this, recent research has moved beyond simple exact-match prefix caching toward more sophisticated systems that enable selective recomputation and segment-level sharing. KVShare introduces a dual-stage high deviation algorithm that conditionally selects small portions of the KV cache to be recomputed during both prefill and decode phases, allowing for accurate cache sharing across requests without sacrificing model accuracy. This approach significantly reduces Time to First Token (TTFT) by avoiding full recomputation while correcting for attention deviations introduced by new tokens. Similarly, SparseX advances this concept by utilizing contiguous token segments as reuse units and employing SparseQ indices to estimate key tokens requiring correction, thereby restoring cross-segment contextual interactions under complex interleaved reuse patterns without needing separate preprocessing stages.

The complexity of cross-request sharing increases substantially in multi-agent and multi-model scenarios where synchronization patterns create massive redundancy. TokenDance specifically targets the AllGather communication pattern inherent in multi-agent applications, where every agent's prompt contains shared output blocks. By performing KV cache reuse over a full round in a single collective step and encoding sibling caches as block-sparse diffs against a master copy, TokenDance achieves substantial compression and supports a higher number of concurrent agents. In heterogeneous environments where different models process inputs with shared context prefixes, DroidSpeak enables KV cache reuse across distributed nodes running different LLMs, provided they share the same architecture. This system selectively recomputes specific layers of the KV cache produced by another model, facilitating cross-model communication with negligible quality loss. Furthermore, ForkKV addresses the memory bottlenecks in multi-LoRA agent workflows by employing a copy-on-write mechanism that physically decouples the KV cache into massive shared components and lightweight agent-specific components, allowing newly forked agents to inherit shared caches efficiently.

Security and isolation remain critical concerns when implementing aggressive cross-request sharing in multi-tenant systems. CacheSolidarity highlights that automatic prefix caching can introduce timing side channels, allowing attackers to infer sensitive information by observing cache hit and miss patterns. Rather than disabling sharing entirely, which sacrifices efficiency, CacheSolidarity monitors reuse across users and selectively isolates prefixes only when suspicious sharing is detected, thereby maintaining high cache reuse rates while mitigating security risks. Complementing this, PRISM co-designs a query-aware scheduler with a demand-aware radix tree to align request admission with exact-prefix KV retention, addressing the gap between scheduling and memory management to reduce tail latency and improve hit rates in the presence of hotspot skew.

Beyond algorithmic innovations, system architecture plays a pivotal role in enabling efficient prefix reuse and cross-request sharing. ObjectCache explores storing KV caches in S3-compatible object storage to remove capacity constraints, co-designing storage protocols to deliver data in the order the GPU consumes it, thus overlapping transfer with compute. Tutti further optimizes this by eliminating CPU intervention from the critical data path between GPU memory and SSDs, utilizing a GPU-centric object store and asynchronous I/O to saturate NVMe bandwidth and reduce GPU stalls. For scenarios where network bandwidth is limited, ShadowServe employs SmartNIC acceleration to perform interference-free prefix caching, separating control and data planes to prevent decompression from interfering with model computation. Additionally, CacheFlow rethinks cache restoration as a multi-dimensional parallel execution problem, introducing a unified 3D parallelism abstraction across tokens, layers, and GPUs to fine-grain overlap recomputation and I/O, significantly reducing TTFT compared to traditional per-request tradeoffs.

The efficacy of these sharing mechanisms often depends on the ability to handle position independence and dynamic context modifications. MEPIC enables chunk-level KV reuse across arbitrary positions by aligning chunk KV to paged storage and fusing Rotary Position Embedding (RoPE) into the attention kernel, making remaining blocks fully shareable and reducing High-Bandwidth Memory usage significantly. Leyline addresses the needs of agentic inference where policies may need to edit or replace spans of cached content; it provides a declarative interface for span

replacement that restores attention math via closed-form RoPE-rotation correction, avoiding full re-prefilling. Meanwhile, CacheBlend tackles the challenge of combining precomputed KV caches from multiple text chunks in Retrieval-Augmented Generation (RAG) by selectively recomputing a small subset of tokens to update reused caches, ensuring generation quality matches full prefill while pipelining recomputation delays with cache retrieval. These diverse approaches collectively demonstrate that effective prefix reuse and cross-request sharing require a holistic integration of algorithmic precision, architectural innovation, and security awareness to maximize throughput and minimize latency in modern LLM serving stacks.

Speculative Decoding and Parallel Generation Mechanisms

Speculative decoding has fundamentally reshaped the landscape of Large Language Model (LLM) inference by addressing the inherent sequential bottleneck of autoregressive generation. The core paradigm involves a "draft-then-verify" strategy where a smaller, faster draft model predicts multiple future tokens, which are subsequently verified in parallel by the larger target model [19]. This approach allows for the simultaneous decoding of multiple tokens per step, effectively increasing the number of tokens generated per second without altering the output distribution of the target model [15]. While early implementations relied on static drafting lengths and rigid verification criteria, recent advancements have shifted towards adaptive, domain-specific, and architecturally novel frameworks that optimize draft diversity, alignment, and verification efficiency.

A primary area of innovation lies in the dynamic adjustment of speculation parameters to accommodate varying input complexities and model uncertainties. Static draft lengths often fail to maximize efficiency across diverse scenarios, leading to suboptimal resource utilization. To address this, researchers have proposed confidence-modulated frameworks that leverage entropy and margin-based uncertainty measures to dynamically adjust the number of speculatively generated tokens at each iteration [30]. Similarly, AdaEDL introduces an entropy-based lower bound on token acceptance probability to enable early stopping of the drafting process, consistently outperforming static methods by significant margins [197]. Further refining this adaptability, DISCO optimizes the speculation lookahead dynamically, while AdaSD employs hyperparameter-free thresholds based on token entropy and Jensen-Shannon distance to adjust generation length and acceptance criteria in real time [39] [131]. These adaptive mechanisms collectively reduce rollback frequency and improve robustness, particularly in high-sampling-temperature scenarios where traditional methods struggle.

Beyond parameter adaptation, significant progress has been made in restructuring the drafting architecture itself to enhance candidate diversity and acceptance rates. Traditional single-path drafting is increasingly being replaced by tree-based and multi-candidate approaches. SpecInfer organizes predictions into a token tree, verifying multiple candidate sequences in parallel to maximize the expected acceptance length [162]. Building on this, OPT-Tree searches for optimal adaptive tree structures that can generate more than ten tokens in a single step if the draft model is sufficiently powerful [246]. To further diversify candidates, Jakiro leverages Mixture of Experts (MoE) to decouple correlations among candidates generated at the same step, while Multi-Candidate Speculative Decoding samples and verifies multiple candidates in batches to improve acceptance rates [296] [129]. Additionally, Ouroboros focuses on generating longer draft phrases rather than individual tokens, parallelizing the drafting process to achieve substantial speedups over standard speculative decoding [248].

The verification phase has also undergone substantial reimagining to mitigate inefficiencies such as the "random rejection" problem and memory bandwidth bottlenecks. EARS addresses random rejections in high-uncertainty scenarios by dynamically adjusting the acceptance threshold based on the target model's predictive uncertainty, thereby increasing throughput with negligible accuracy loss [EARS: Efficient Adaptive Rejection Sampling for Accelerating Speculative Decoding in Large Language Models]. On the system level, CARD decouples drafting and verification into a "query-and-correct" paradigm, allowing the target model to concurrently rectify the draft model's direction and approach the draft model's inference speed [87]. Furthermore, Quasar tackles the memory wall during verification by employing low-bit quantization specifically for the verification stage, halving memory traffic while preserving logit distribution fidelity [111].

Specialized applications and hardware constraints have driven the development of domain-specific and self-speculative variants. For multimodal models, MSD decouples text and visual token processing in the draft model, while HIPPO employs holistic-aware parallel decoding for video-LLMs to preserve semantic information during pruning [54] [113]. In resource-constrained environments, self-speculative methods like Cassandra and SparseSpec utilize the target model itself for drafting via pruning or sparse attention, eliminating the need for auxiliary models [195] [104]. Moreover, hardware-accelerated approaches like HADES and ReRAM-based accelerators integrate speculative decoding directly into chip design to optimize energy and latency [89] [156]. Despite these advances, challenges remain regarding privacy risks, where token generation patterns can leak user inputs, and the need for robust distributed frameworks to handle communication overhead in collaborative settings [98] [125]. The evolution from simple draft-verify paradigms

to these sophisticated, adaptive, and integrated systems underscores the critical role of speculative decoding in enabling scalable and efficient LLM deployment.

Dynamic Tree Construction and Verification

The structural topology of draft tokens has emerged as a decisive factor in maximizing the efficiency of speculative decoding, marking a significant departure from early static, linear draft sequences. While initial approaches relied on fixed heuristic structures that failed to adapt to varying context complexities, contemporary research demonstrates that adaptive tree structures can significantly increase the mathematical expectation of acceptance length per decoding step. [246] proposes an algorithm that dynamically searches for the optimal tree configuration, allowing for the generation of more than ten tokens in a single step when the draft model possesses sufficient capability. This method achieves speedups of up to 3.2x over standard autoregressive decoding by tailoring the tree shape to the specific probability distributions of the model at each step, thereby overcoming the limitations of rigid, pre-defined draft lengths.

However, the optimization of tree structures is not merely an algorithmic challenge; it is inextricably linked to underlying system variables and hardware constraints. A critical divergence in recent literature highlights that approaches focusing solely on model probabilities often neglect crucial inference costs. [228] introduces CAST, a methodology that explicitly incorporates GPU configurations and batch sizes into the tree construction logic. CAST argues that ignoring these system-level variables leads to suboptimal performance in diverse hardware environments, a gap left by methods like EAGLE-2 and EAGLE-3. By dynamically refining tree structures based on inference costs, CAST achieves speedups of up to 5.2x, outperforming existing state-of-the-art techniques by 5% to 20%. This underscores a growing consensus that effective tree construction must be co-designed with hardware awareness, moving beyond purely probabilistic optimizations to include cost-aware scheduling.

Complementing structural adaptation, mechanisms for dynamically controlling the depth and breadth of the draft tree have become essential for preventing computational waste. Static draft length configurations often negatively impact performance, particularly in scenarios with high variance in token acceptance. [197] proposes a training-free criterion that utilizes the entropy of drafted logits to approximate a lower bound on acceptance probability. This allows for the early stopping of the drafting process, ensuring that resources are not wasted generating tokens unlikely to be accepted. Similarly, [131] introduces adaptive thresholds based on token entropy and Jensen-Shannon distance to dynamically adjust generation length and acceptance criteria without requiring hyperparameter tuning. These methods contrast sharply with static configurations, offering robustness in high-sampling-temperature scenarios where acceptance rates fluctuate wildly.

The theoretical underpinnings of tree-based verification have also been refined to support more complex topologies and parallel verification mechanisms. [162] leverages small speculative models to organize predictions into a token tree, where nodes represent candidate sequences verified in parallel by the target model. This tree-based parallel decoding mechanism significantly reduces end-to-end latency compared to incremental decoding. Furthermore, [239] provides a principled understanding of speculative decoding through optimal transport theory, generalizing the method to allow for a set of candidates at the token level. This formulation enables the computation of an optimal draft selection algorithm that improves acceptance probability, offering a theoretical basis for the efficacy of tree-based approaches over linear chains.

Despite the advantages of tree structures, challenges remain regarding memory overhead and the sequential bottlenecks inherent in the "draft-then-verify" paradigm. [87] critiques the strict sequential execution of drafting and verification, proposing a "query-and-correct" paradigm that decouples these processes using a shared cache. This allows the target model to rectify the draft model's direction concurrently, mitigating the inefficiency where a single rejection discards all subsequent candidates. Additionally, [248] focuses on generating draft phrases to parallelize the drafting process itself, lengthening drafts in a training-free manner to achieve speedups of up to 3.9x over vanilla decoding.

The interaction between tree-based speculation and batching strategies further complicates the optimization landscape, as the optimal speculation length is dependent on batch size. [8] observes this dependency and proposes an adaptive strategy that selects the optimal length for different batch configurations. This finding aligns with [11], which notes that speculative decoding can degrade performance in high-load, compute-bound environments due to verification overhead. Nightjar proactively disables speculation when it is no longer beneficial and offloads the draft model to reclaim KV-cache memory for larger batch sizes, illustrating the critical trade-off between aggressive tree expansion and memory availability for batching.

Hardware-specific constraints also dictate the viability of dynamic tree construction, particularly in resource-constrained environments. [74] adapts tree-based methods for consumer GPUs with limited VRAM, utilizing parameter offloading to build "cache" trees that can validate up to 20 tokens per iteration. This contrasts with datacenter-focused approaches, emphasizing that tree depth and breadth must be calibrated to the specific memory hierarchy of the deployment environment. Similarly, [156] presents an accelerator design featuring adaptive parallel speculative decoding that switches between short and long draft lengths based on verification feedback, optimizing resource utilization in edge computing scenarios.

Finally, the robustness of tree structures across different model architectures and tasks remains a focal point for ensuring consistent acceleration. [123] organizes token sources into a hierarchical framework based on temporal locality, ensuring consistent acceleration across diverse tasks without significant fine-tuning. This approach addresses the inconsistency often found in drafting strategies that rely heavily on specific task distributions. Meanwhile, [202] enhances LLMs with semi-autoregressive generation capabilities using efficient tree-based decoding, demonstrating that architectural modifications can further synergize with tree verification to achieve speedups of 2.7x on large models like LLaMA-2-70B-Chat. Collectively, these works establish that dynamic tree construction is not a monolithic solution but a multifaceted strategy requiring adaptation to system variables, hardware constraints, and workload characteristics to realize its full potential in accelerating LLM inference.

Draft Model Optimization and Diversity

The efficacy of speculative decoding is fundamentally contingent upon the quality, diversity, and architectural efficiency of the draft model employed to generate candidate tokens. A primary limitation in conventional speculative decoding frameworks is the restricted diversity of candidate tokens, often stemming from the fact that multiple candidates at a single decoding step are derived from identical internal representations. This homogeneity limits the probability of acceptance by the target model, particularly in scenarios requiring complex reasoning or creative generation. To address this lack of diversity, Jakiro: Boosting Speculative Decoding with Decoupled Multi-Head via MoE introduces a novel approach leveraging Mixture of Experts (MoE) within the draft model. By decoupling multi-head predictions through independent experts, Jakiro generates significantly more diverse candidate sets, effectively breaking the correlation among candidates that plagues standard tree-based sampling. This architectural modification not only improves prediction accuracy but also enhances the overall inference speedup by increasing the acceptance rate of drafted tokens. Complementing these structural innovations, Tandem Transformers for Inference Efficient LLMs proposes a unique architecture where a small autoregressive model attends to the richer representations of a large model to enhance predictive accuracy before verification. This "Tandem" approach yields higher per-block acceptance rates compared to vanilla secondary models, suggesting that tighter coupling of representations between draft and target models can significantly boost diversity and accuracy.

Beyond architectural changes, optimizing the draft model's capacity to predict longer sequences or multiple candidates simultaneously is critical for performance. Multi-Candidate Speculative Decoding proposes sampling multiple candidates from a draft model and organizing them into batches for parallel verification, demonstrating that increasing the breadth of candidates can consistently outperform standard speculative decoding across various datasets. Similarly, OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure argues that fixed heuristic draft structures are suboptimal. Instead, it proposes an algorithm to construct adaptive and scalable draft trees that maximize the mathematical expectation of acceptance length, capable of generating over ten tokens in a single step if the draft model is sufficiently powerful. These approaches highlight a consensus that expanding the search space of the draft model-whether through width via multi-candidate strategies or depth via tree structures-is critical for performance, provided the verification overhead does not negate the gains. Furthermore, Ouroboros: Generating Longer Drafts Phrase by Phrase for Faster Speculative Decoding introduces a training-free method to generate draft phrases, lengthening drafts efficiently and achieving significant speedups without fine-tuning.

However, simply increasing the number of candidates or the length of the draft sequence introduces new challenges regarding computational overhead and memory bandwidth, particularly for the draft model itself. CORAL: Learning Consistent Representations across Multi-step Training with Lighter Speculative Drafter identifies the Language Modeling (LM) head as a major bottleneck in draft model inference speed. To mitigate this, CORAL introduces a weight-grouping mechanism that selectively activates a subset of LM head parameters during inference, alongside a Cross-Step Representation Alignment method to improve training consistency. This focus on reducing the draft model's latency is crucial because, as noted in Quasar: Quantized Self-Speculative Acceleration for Rapid Inference via Memory-Efficient Verification, the verification phase often becomes the new bottleneck once drafting overhead is minimized. Quasar addresses this by employing low-bit quantization specifically for the verification stage, preserving logit distribution fidelity while halving memory traffic, thereby creating a more balanced system where neither drafting nor verification dominates the latency profile. Additionally, FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling optimizes draft candidate selection through vocabulary space compression, reducing LM Head computation overhead significantly for large-vocabulary models.

The optimization of draft models also extends to the alignment between the draft and target models, especially in heterogeneous or multi-target deployment scenarios. S2D: Sorted Speculative Decoding For More Efficient Deployment of Nested Large Language Models addresses the complexity of serving multiple target models by introducing a sorted speculative decoding mechanism that efficiently matches draft models to various targets without requiring customized training for each pair. In distributed settings, Fast Collaborative Inference via Distributed Speculative Decoding proposes a sparsify-then-sample strategy to reduce uplink communication overhead while maintaining acceptance rates, demonstrating effectiveness for scalable, communication-efficient distributed LLM inference. Moreover, Context-Aware Assistant Selection for Improved Inference Acceleration with Large Language

Models frames the selection of draft models as a contextual bandit problem, showing that policies trained on output alignment can accelerate performance across multiple domains even without prior knowledge of the draft models' internal construction.

Despite these advancements, contradictions and trade-offs remain regarding the optimal strategy for candidate generation under varying uncertainty conditions. While Jakiro: Boosting Speculative Decoding with Decoupled Multi-Head via MoE and Multi-Candidate Speculative Decoding advocate for generating more diverse candidates to improve acceptance, EARS: Efficient Adaptive Rejection Sampling for Accelerating Speculative Decoding in Large Language Models points out that in high-uncertainty scenarios, plausible candidates are often rejected due to rigid, context-independent random thresholds rather than poor quality. EARS proposes dynamically adjusting the acceptance threshold based on the target model's predictive uncertainty, effectively relaxing criteria when the model is unsure. This suggests that draft model optimization must be viewed holistically: improving the diversity of the draft output must be paired with adaptive verification mechanisms to fully realize efficiency gains. Additionally, AdaEDL: Early Draft Stopping for Speculative Decoding of Large Language Models via an Entropy-based Lower Bound on Token Acceptance Probability reinforces the need for adaptability by proposing an entropy-based criterion for early stopping, arguing that static draft lengths fail to account for the variance in token acceptance, thereby wasting computation on low-probability continuations. Similarly, Confidence-Modulated Speculative Decoding for Large Language Models leverages entropy and margin-based uncertainty measures to dynamically adjust both the number of speculatively generated tokens and the verification process, reducing rollback frequency and improving resource utilization.

Finally, the scope of draft model optimization is expanding beyond text-only domains and standard hardware constraints. Speculative Decoding Reimagined for Multimodal Large Language Models and HIPPO: Accelerating Video Large Language Models Inference via Holistic-aware Parallel Speculative Decoding demonstrate that draft models for multimodal tasks require specialized handling of visual and textual tokens. HIPPO, for instance, employs a semantic-aware token preservation method to maintain visual semantics during pruning, ensuring that the draft model retains sufficient context to generate high-quality candidates for video-LLMs. In resource-constrained environments, Cassandra: Enabling Reasoning LLMs at Edge via Self-Speculative Decoding constructs a high-performance, training-free draft model through fine-grained data selection and mantissa truncation, achieving substantial performance gains on consumer-grade devices. These developments indicate that future draft model optimizations must be domain-specific and hardware-aware, leveraging the unique characteristics of the input modality and the deployment environment to maximize the diversity and relevance of generated candidates.

Conclusion

The literature on efficient Large Language Model (LLM) serving reveals a fundamental paradigm shift from compute-bound to memory-bound constraints, driven by the linear growth of Key-Value (KV) caches and the widening gap between processor speed and memory bandwidth. Addressing this "memory wall" requires a holistic co-design strategy that integrates algorithmic compression, hierarchical memory management, and hardware-aware scheduling. While compression techniques such as quantization and sparsity offer significant footprint reductions, they introduce complex trade-offs between accuracy and latency, particularly when handling activation outliers or long-context dependencies. Similarly, hierarchical offloading strategies expand effective capacity but risk saturating interconnect bandwidth, necessitating GPU-centric I/O stacks and emerging high-bandwidth interconnects to prevent severe underutilization of compute resources.

A critical tension exists between static optimization policies and the dynamic, stochastic nature of real-world workloads. The field is increasingly moving away from rigid heuristics toward adaptive, learning-based controllers that can dynamically adjust speculation lengths, precision levels, and cache eviction strategies in response to fluctuating demand and hardware states. This evolution is evident in the transition from simple draft-verify paradigms to sophisticated tree-based speculative decoding and conversation-level scheduling, which better accommodate the asymmetry between prefill and decode phases. Furthermore, the rise of specialized architectures, including recurrent models and decoupled tandem transformers, suggests that future efficiency gains will depend not only on optimizing existing Transformer pipelines but also on reimagining the underlying computation graphs to inherently reduce memory dependencies.

Ultimately, achieving scalable and sustainable LLM deployment demands a unified approach that bridges the gap between theoretical algorithmic improvements and practical system constraints. Future advancements must prioritize standardized evaluation metrics that account for energy efficiency, tail latency, and cost-per-token across heterogeneous hardware environments, from datacenter superchips to resource-constrained edge devices. By synthesizing innovations in quantization, speculative execution, and predictive scheduling, the next generation of serving systems will be better equipped to navigate the competing demands of throughput, latency, and model fidelity, enabling the widespread adoption of large-scale AI applications.